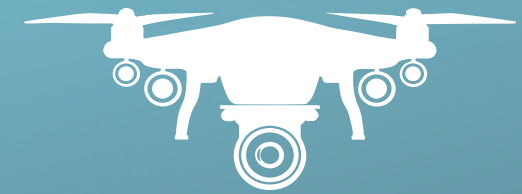


Eqaab Drone



OUTLINE

01 Introduction

02 Problem Statement

03 Objectives

04 Scope

05 Literature Review

06 System Design

07 Simulation and Result

07 Conclusion

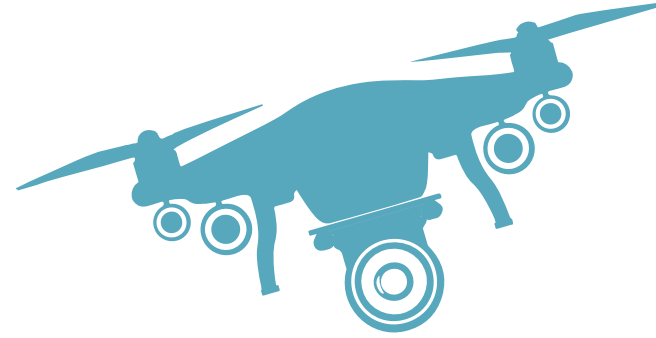




DRONE

“INTRODUCTION

Overview



- ❑ UAVs (drones) are modern tools for wide-area surveillance and monitoring.
- ❑ They provide flexible, mobile, bird's-eye views with real-time video.
- ❑ AI-powered computer vision enables automatic detection and recognition of vehicles and people.
- ❑ Drone-based surveillance supports applications in traffic monitoring, public safety, and security zones.





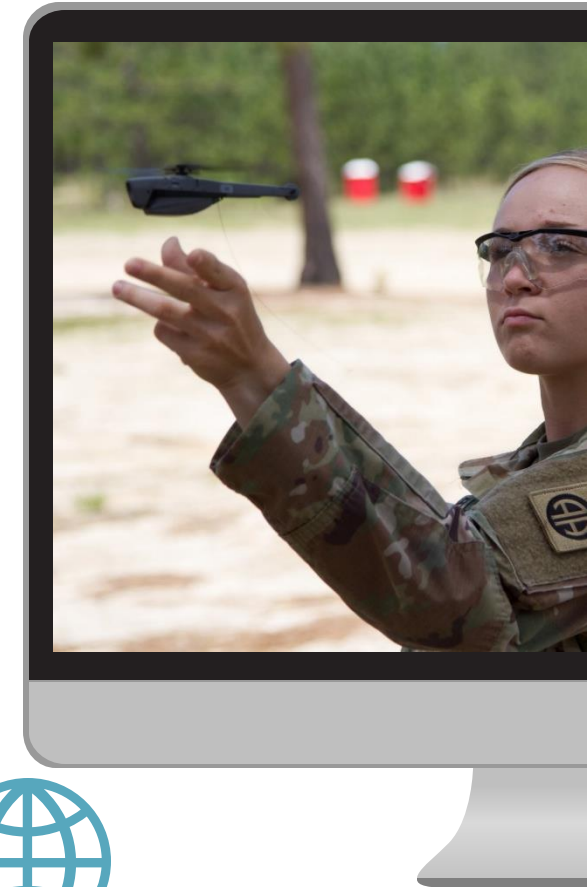
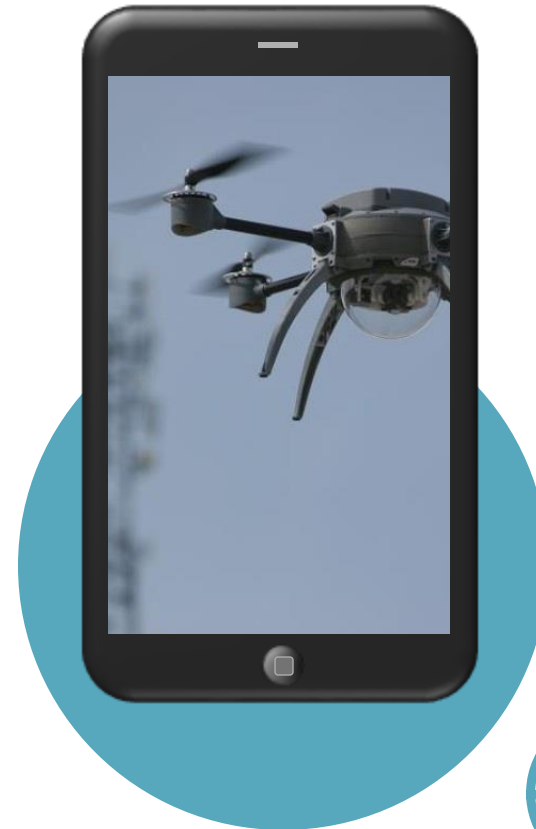
DRONE

**“ PROBLEM
STATEMENT**

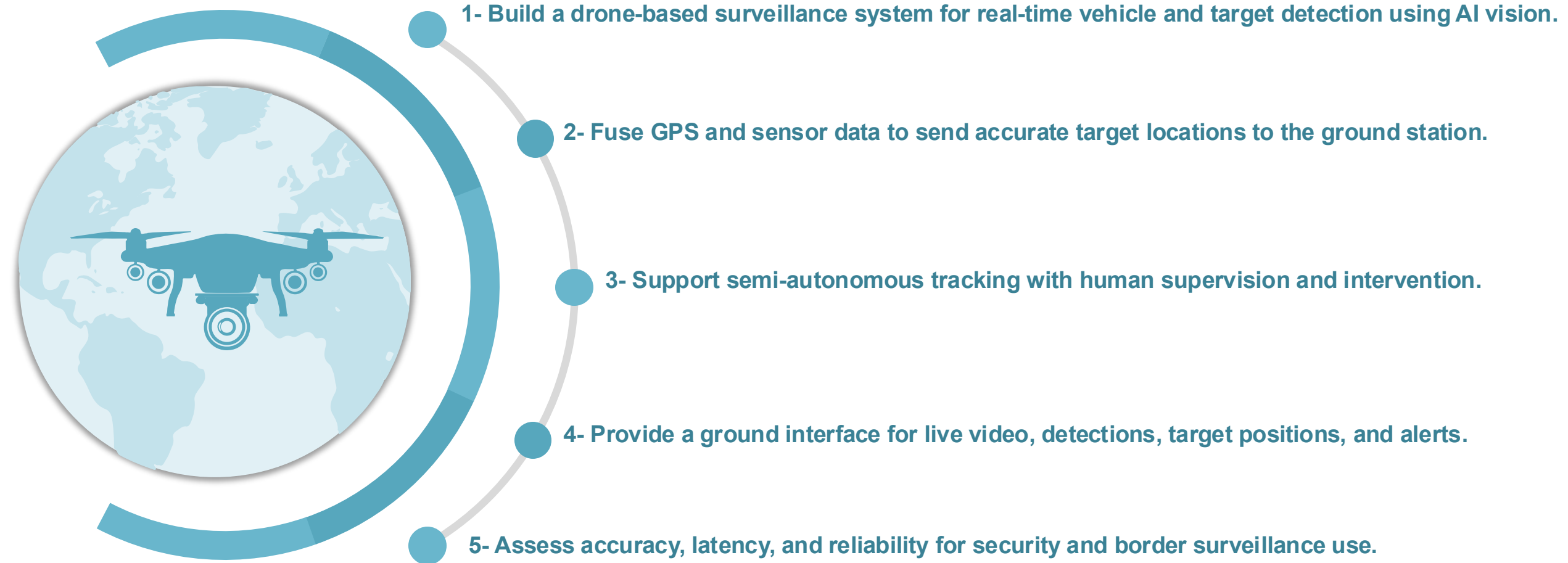
PROBLEM STATEMENT



- ❑ Large high-security areas still have blind spots with undetected vehicles or intruders.
- ❑ Fixed cameras and ground sensors lack full coverage in complex or remote terrains.
- ❑ Heavy reliance on human operators leads to fatigue, missed threats, and slow response.
- ❑ Limited tracking and location logging weaken incident review and lessons learned.



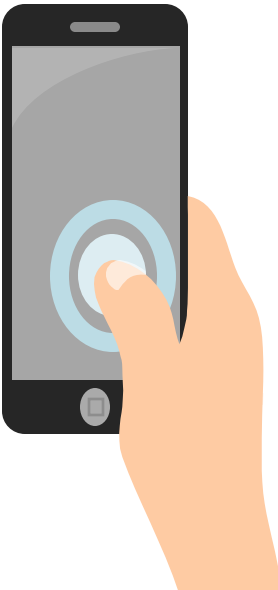
OBJECTIVES



Scope



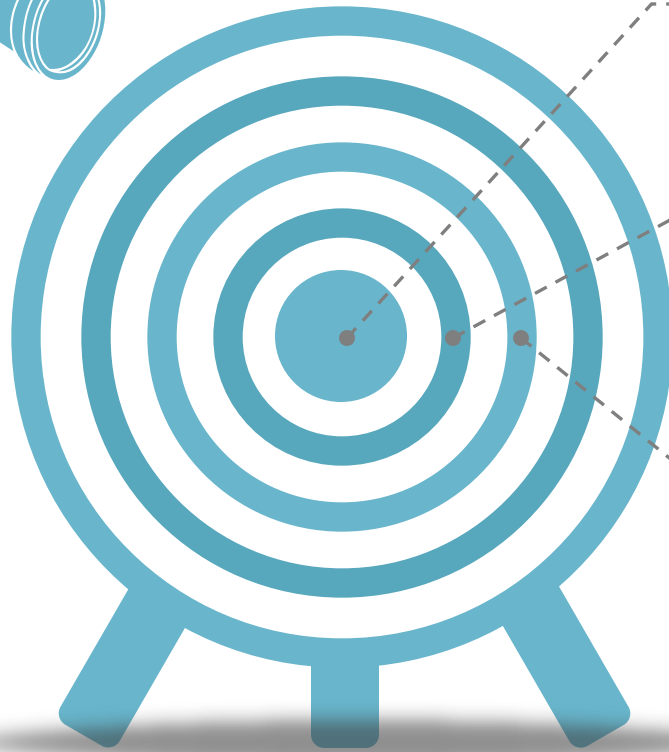
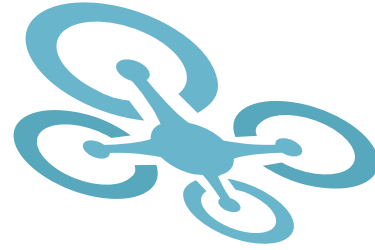
This project develops and tests a single drone-based intelligent surveillance prototype, including hardware integration, AI-based detection/tracking, and a simple ground control interface for security and border surveillance.



L i t e r a t u r e R e v i e w



Literature Review

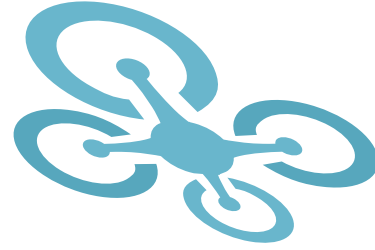
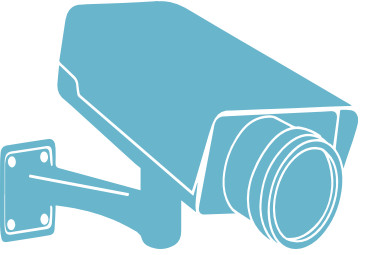


MADRL framework to coordinate multiple UAVs for reliable, energy-efficient city surveillance.

YOLO-SRMX, a lightweight infrared UAV detector that improves small-target accuracy with much lower computation.

YOLO-UD, an enhanced YOLO detector for UAV imagery that boosts dense small-object detection while keeping real-time speed.

Literature Review



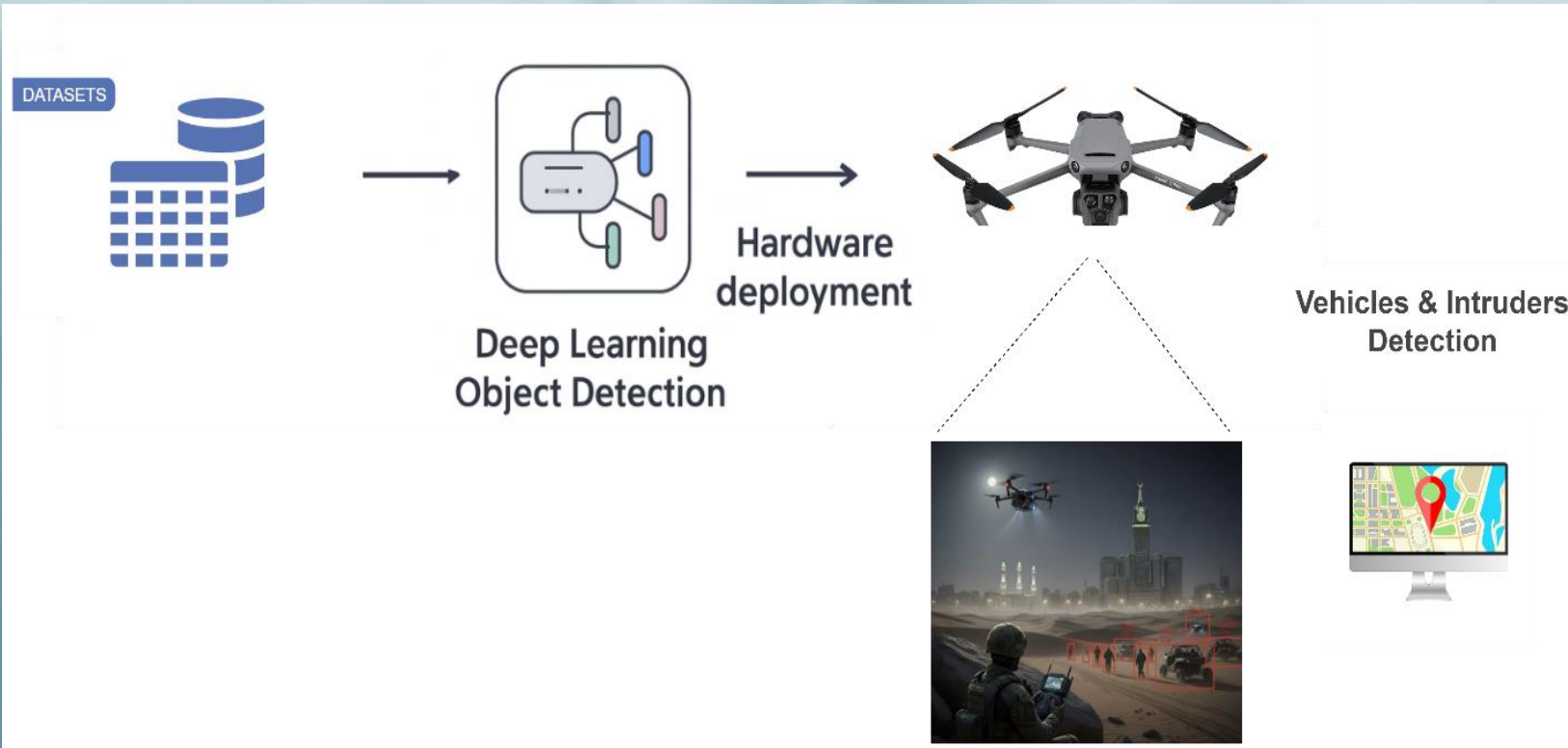
Paper	Detect vehicles	Detect Humans	Camera	Real-time tracking	object	Autonomous	AI	GPS	UI
MADRL	X	X	X	X		✓	✓	X	X
YOLO-SRMX	✓	✓	✓	X		X	✓	X	X
YOLO-UD	✓	✓	X	X		X	✓	X	X
Eqaab Drone	✓	✓	✓	✓		✓	✓	✓	✓

SYSTEM DESIGN



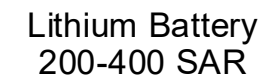
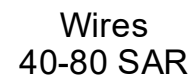
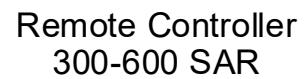
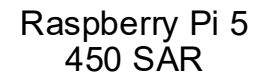
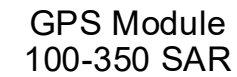
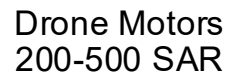
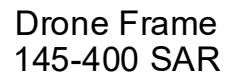
SYSTEM MODEL

- ❑ Train a deep-learning detector on labeled vehicle and intruder datasets, then deploy it on the UAV.
- ❑ UAV runs real-time detection and sends target locations to the ground station for map-based monitoring.



Hardware Specifications





Software Specifications



Software Specifications



Flight control

- Raspberry Pi 5 with PX4 and ROS, validated in Gazebo/MATLAB.

AI & vision

- OpenCV + YOLOv8 on TensorFlow for real-time detection and tracking.

Development

- Python/C++, VS Code & Colab, with Git/GitHub for version control.

Source Code



```
1 import cv2
2 import numpy as np
3 import requests
4 from ultralytics import YOLO
5 from io import BytesIO
6
7 # --- Settings ---
8 # Image URL to process
9 IMAGE_URL = "https://ultralytics.com/images/bus.jpg"
10
11 # Classes to translate or display (only these will be drawn)
12 TARGET_CLASSES = {
13     "person": "person",
14     "car": "car",
15     "truck": "truck",
16     "bus": "bus",
17     "motorbike": "motorcycle",
18     "bicycle": "bicycle"
19 }
20
21 # Output file name
22 OUTPUT_FILE = "result_detected.jpg"
23 Original_FILE = "Original_img.jpg"
24
25 # --- Download image from URL ---
26 def download_image(url: str) -> np.ndarray:
27     """Downloads an image from a URL and returns it as an OpenCV BGR array."""
28     response = requests.get(url, timeout=15)
29     response.raise_for_status()
30     img_arr = np.asarray(bytearray(response.content), dtype=np.uint8)
31     img = cv2.imdecode(img_arr, cv2.IMREAD_COLOR)
32     if img is None:
33         raise ValueError("Failed to load or decode the image.")
34     return img
35
36
37 # --- Run YOLO detection and draw boxes ---
38 def detect_and_draw(img: np.ndarray,
39                     model_name: str = "yolov8n.pt",
40                     conf_threshold: float = 0.35) -> np.ndarray:
41     """
42     Runs YOLOv8 detection on an image and draws bounding boxes
43     around the detected objects (only those listed in TARGET_CLASSES).
44     """
45     model = YOLO(model_name)
46
47     # YOLO expects RGB
48     img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
49
```

1

```
50
51 # Run prediction
52 results = model.predict(source=img_rgb, conf=conf_threshold, imgsz=640, verbose=False)
53 r = results[0]
54
55 boxes = r.boxes
56 if boxes is None or len(boxes) == 0:
57     print("No objects detected at the selected confidence threshold.")
58     return img
59
60 h, w = img.shape[:2]
61 names = model.model.names # class index -> class name mapping
62
63 output = img.copy()
64
65 for box in boxes:
66     xyxy = box.xyxy[0].cpu().numpy() # bounding box coordinates
67     conf = float(box.conf[0].cpu().numpy())
68     cls_idx = int(box.cls[0].cpu().numpy())
69     cls_name = names.get(cls_idx, str(cls_idx))
70
71     # Only draw selected classes
72     if cls_name not in TARGET_CLASSES:
73         continue
74
75     x1, y1, x2, y2 = map(int, xyxy)
76
77     # Draw rectangle
78     thickness = max(1, round(0.002 * (w + h) / 2))
79     cv2.rectangle(output, (x1, y1), (x2, y2), (0, 255, 0), thickness)
80
81     # Label text
82     label = f"{TARGET_CLASSES[cls_name]} {conf:.2f}"
83
84     # Text placement
85     font = cv2.FONT_HERSHEY_SIMPLEX
86     font_scale = 0.6
87     (text_w, text_h), baseline = cv2.getTextSize(label, font, font_scale, 1)
88     text_x = x1
89     text_y = y1 - 10 if y1 - 10 > text_h else y1 + text_h + 10
90
91     # Background rectangle
92     cv2.rectangle(
93         output,
94         (text_x, text_y - text_h - baseline),
95         (text_x + text_w, text_y + baseline),
96         (0, 255, 0),
97         cv2.FILLED
98
```

2

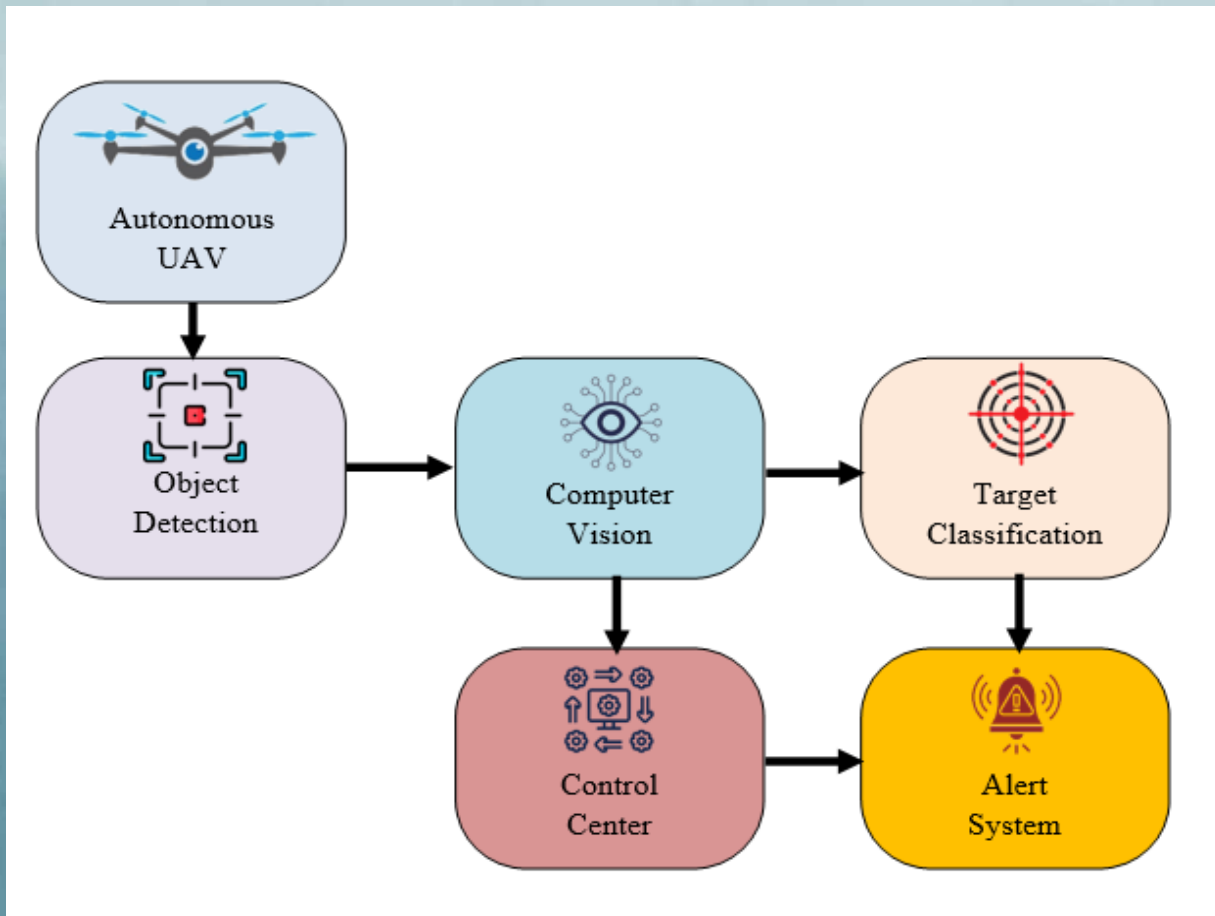
```
99
100     cv2.FILLED
101 )
102
103     # Draw text
104     cv2.putText(output, label, (text_x, text_y), font, font_scale,
105                 (0, 0, 0), thickness=1, lineType=cv2.LINE_AA)
106
107 return output
108
109 # --- Main script ---
110 if __name__ == "__main__":
111     try:
112         print("Downloading image...")
113         img = download_image(IMAGE_URL)
114         cv2.imshow("original", img)
115         cv2.imwrite(Original_FILE, img)
116
117         print("Running YOLO detection...")
118         out_img = detect_and_draw(img, model_name="yolov8n.pt", conf_threshold=0.35)
119
120         cv2.imwrite(OUTPUT_FILE, out_img)
121         print(f"Saved result to: {OUTPUT_FILE}")
122
123         # To display the result locally:
124         cv2.imshow("Detections", out_img)
125         cv2.waitKey(0)
126         cv2.destroyAllWindows()
127
128     except Exception as e:
129         print("Error:", e)

```

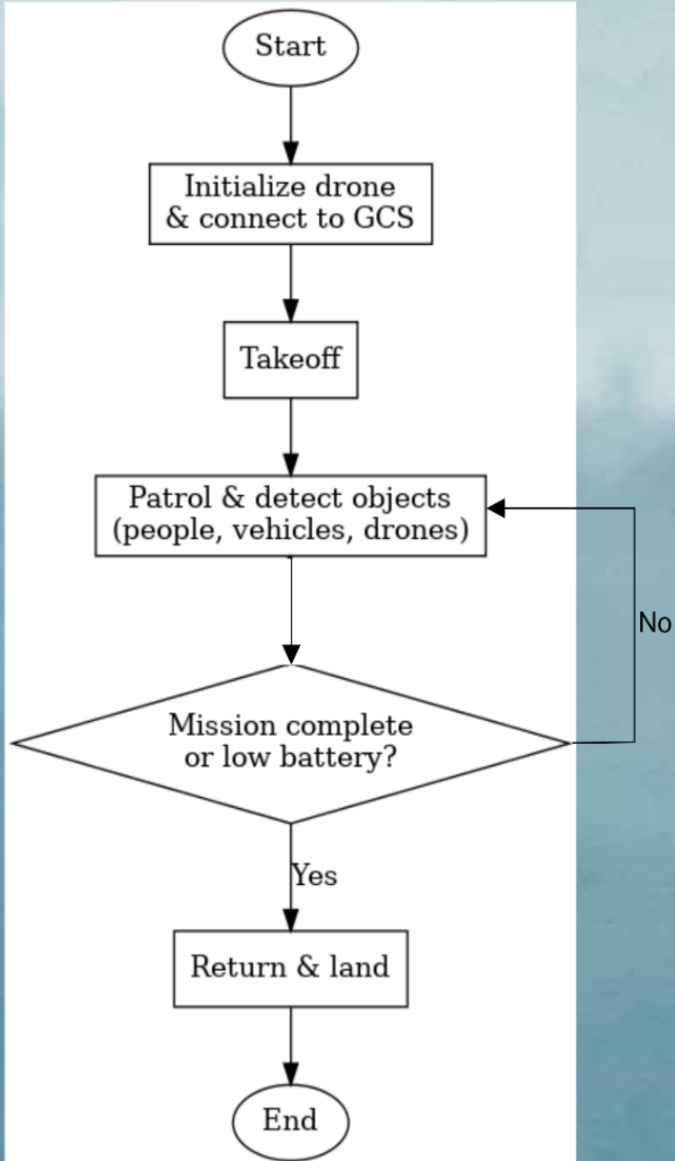
3

System Block Diagram

- ❑ UAV-based AI surveillance system that detects and classifies targets in real time and sends alerts and locations to a control center.



System Flowchart

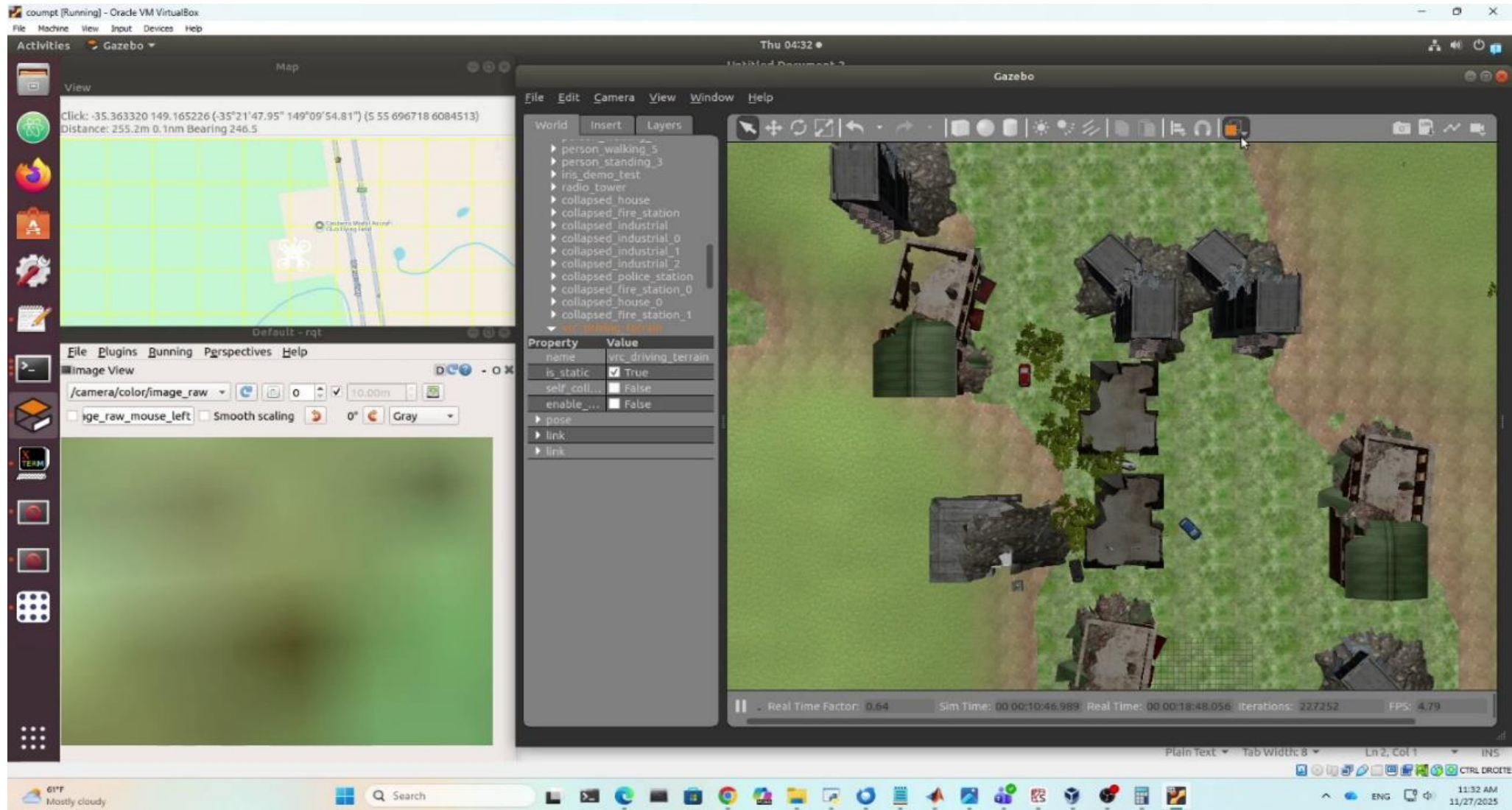


Drone connects to GCS, takes off, patrols while detecting targets, then returns and lands automatically when the mission ends or battery is low.

SIMULATION



Simulation environment



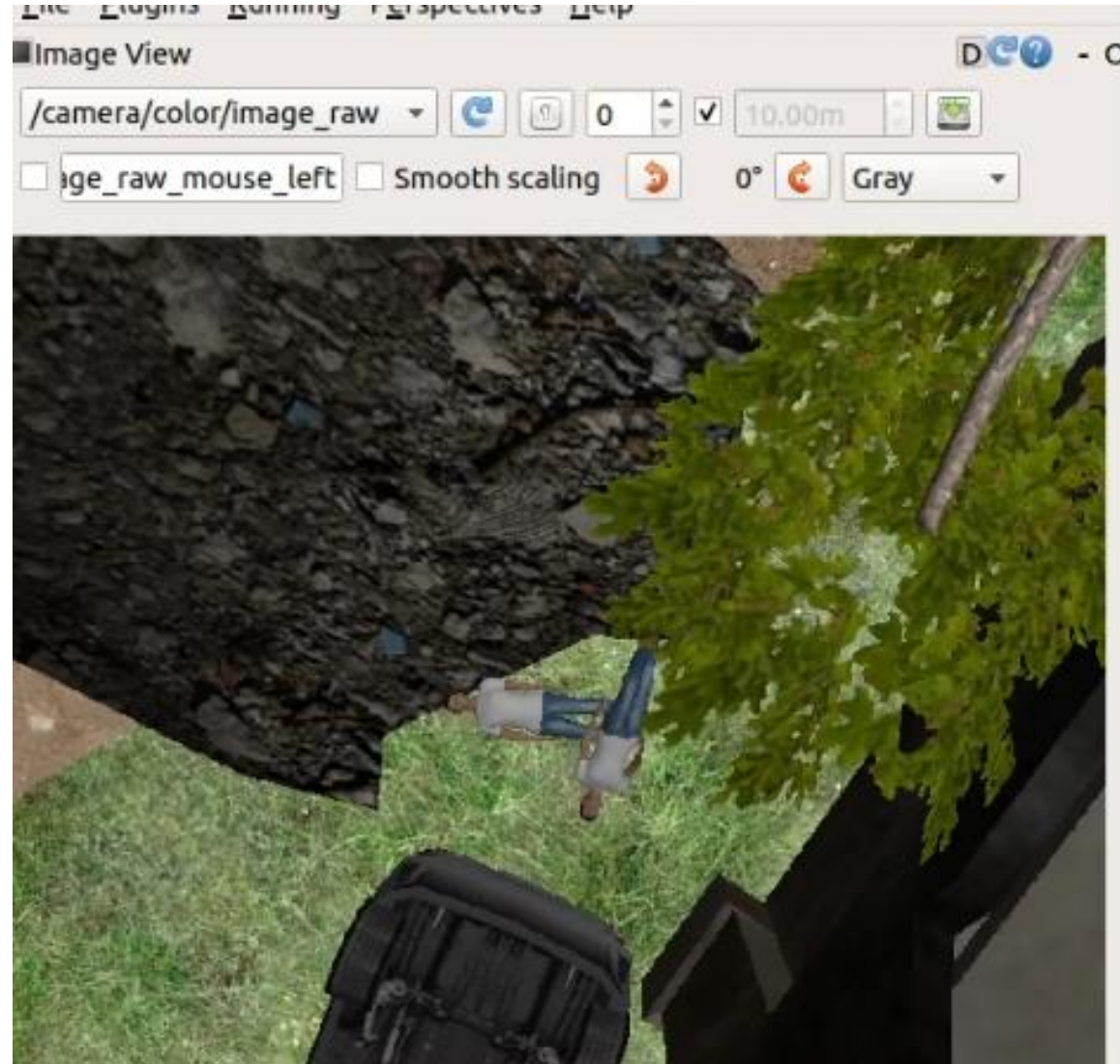
A map of the turn path during the simulation



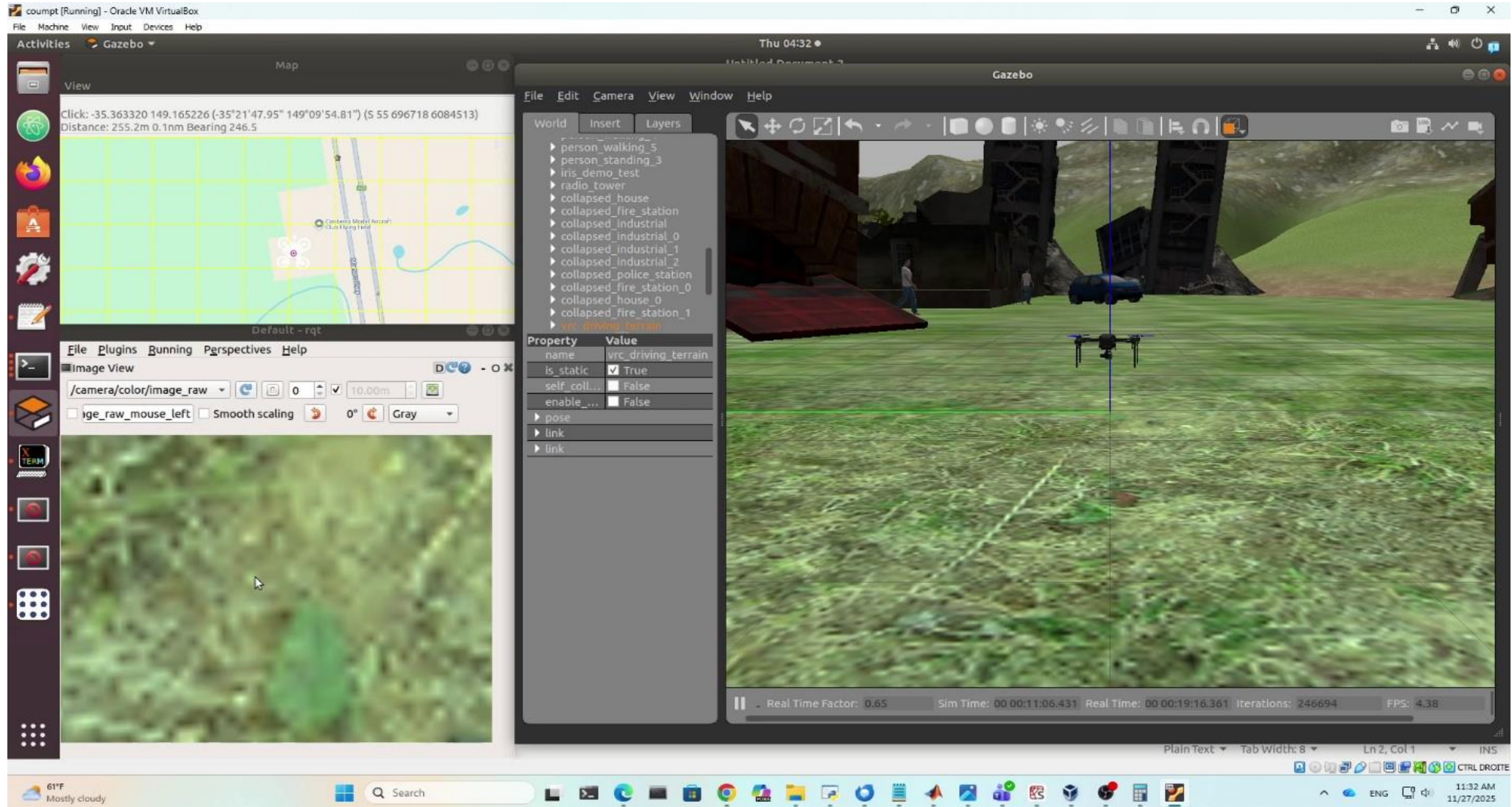
Click: -35.363320 149.165226 (-35°21'47.95" 149°09'54.81") (S 55 696718 6084513)
Distance: 255.2m 0.1nm Bearing 246.5



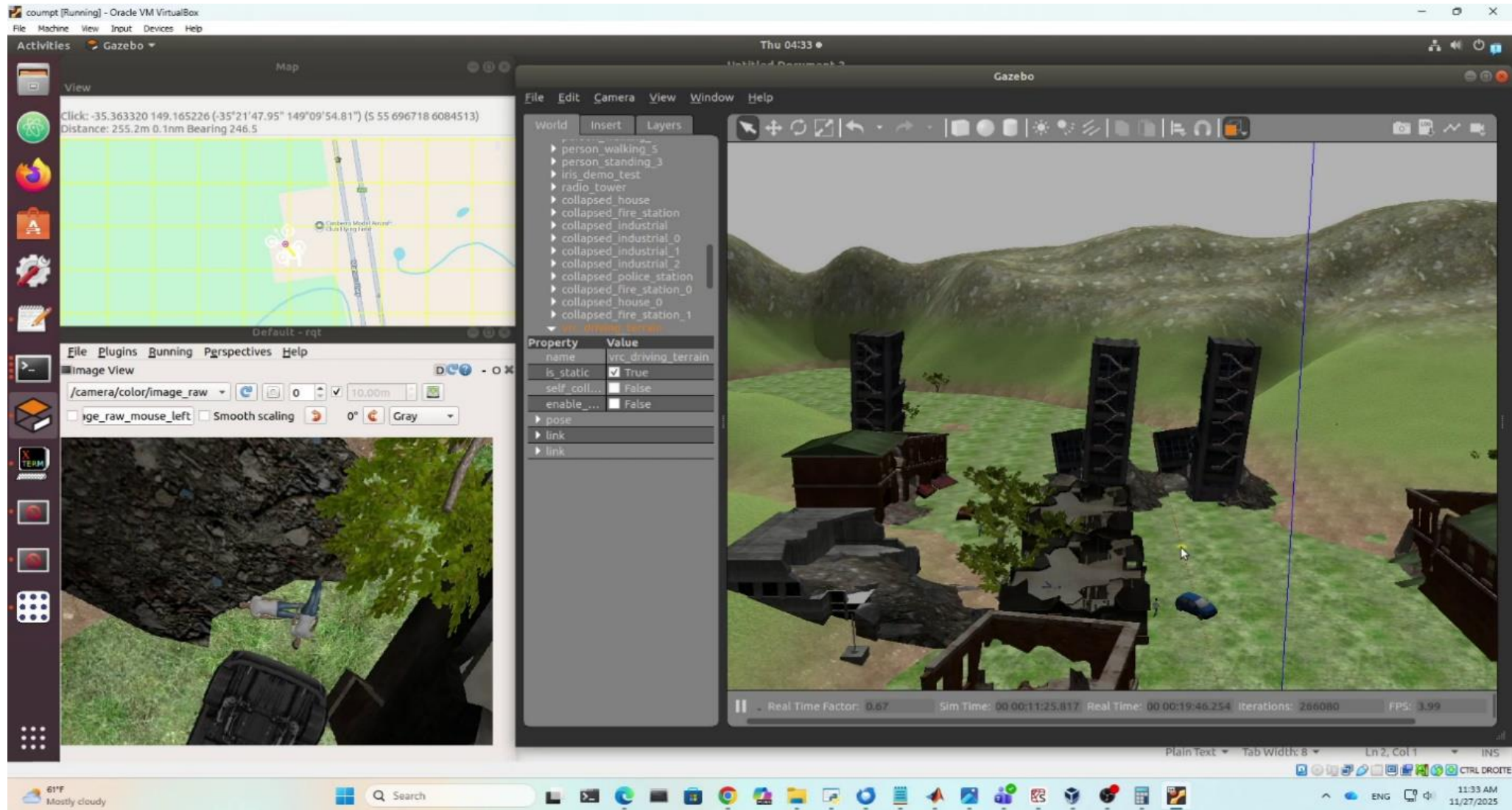
A picture taken from the drone while carrying out the mission



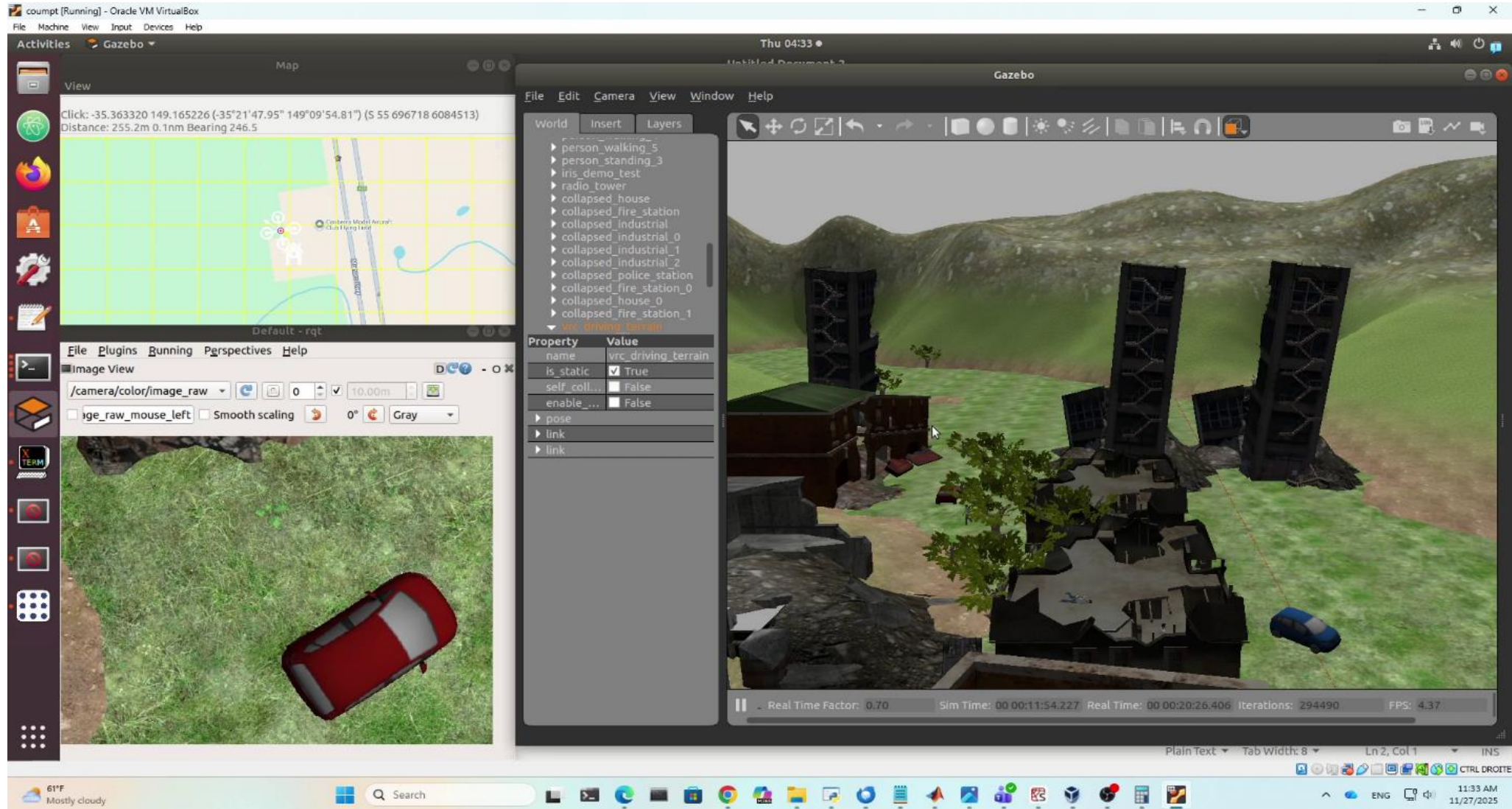
The first moments of the drone taking off



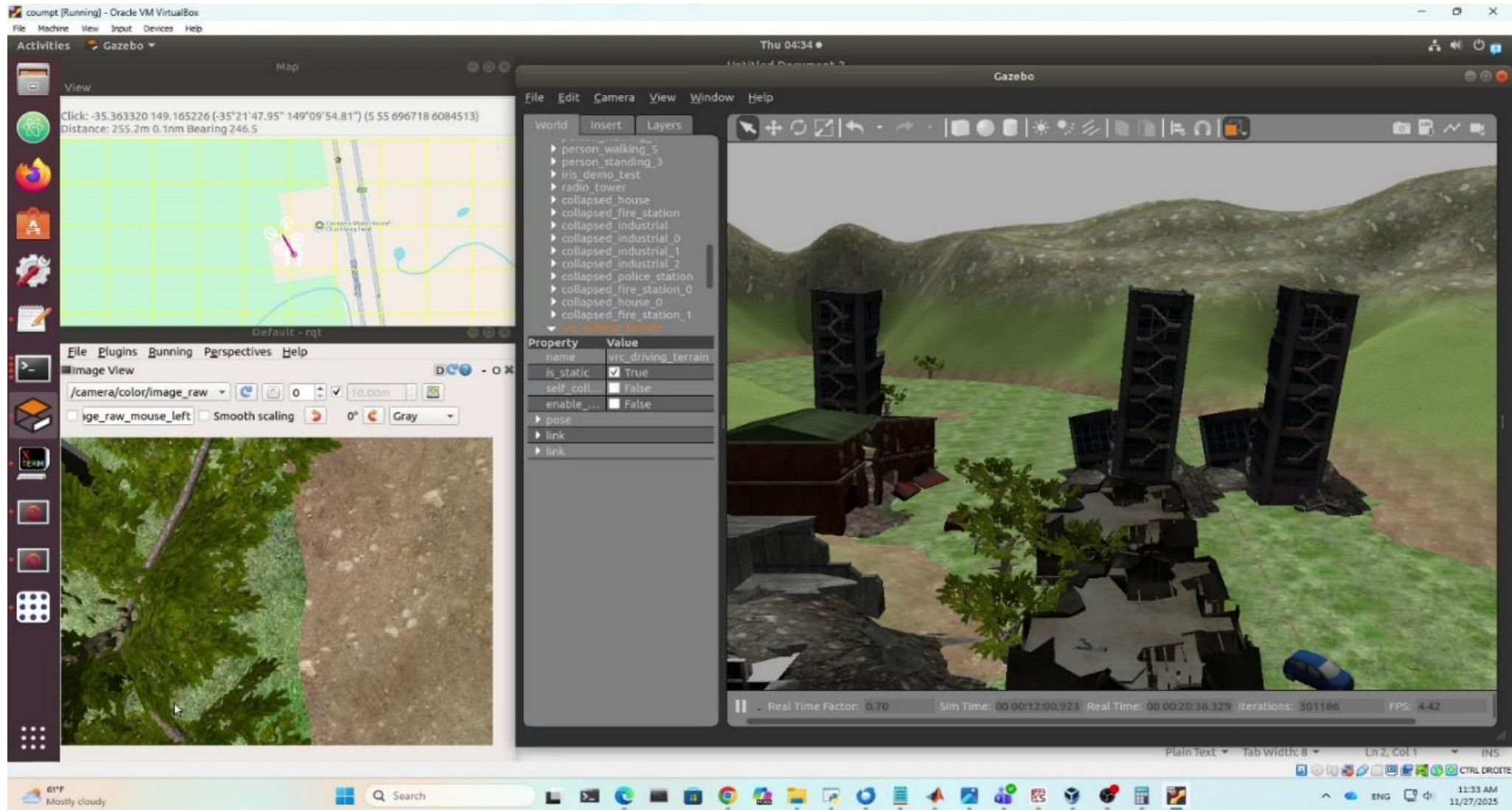
Broadcasts video from drone



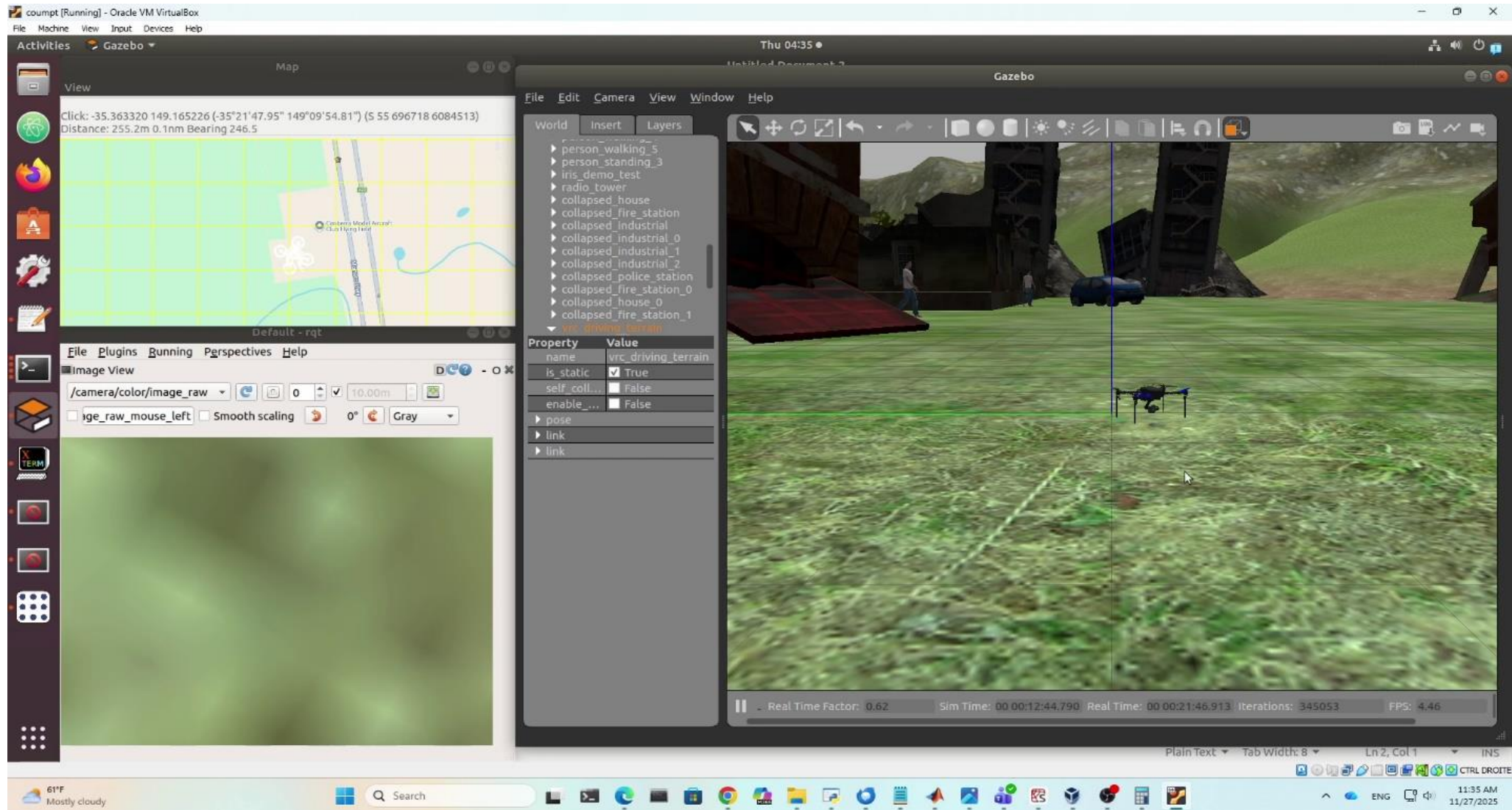
High-quality images and videos from the drone



Finishing the drone's missions and returning home



The drone returns home and carries out the landing



Results



constraint



constraint



- ❑ **Data Availability:** We faced a significant shortage of open-source datasets for 'drone-to-drone' detection. Finding labeled images of drones captured from *another* flying drone is extremely rare.
- ❑ **Economic & Manufacturing:** We initially considered 3D printing the drone structure. However, we found that printing a frame durable enough for this size is highly expensive and time-consuming.
- ❑ **Computational Power:** Running advanced AI models like YOLOv8 on a small computer (Raspberry Pi 5) is challenging.
- ❑ **Legal Regulations:** Operating drones is subject to strict aviation regulations. We are constrained by flight zones and privacy laws, which dictate where and how we can perform our physical testing.





DRONE

“ CONCLUSION ”

CONCLUSION



- ❑ Built and tested the Eqaab prototype with PX4, GPS, camera, and onboard AI for autonomous surveillance.
- ❑ Demonstrated that an AI-enabled UAV can extend coverage and improve real-time situational awareness.
- ❑ **Future work:** large-scale outdoor trials, thermal/IR sensing, better datasets, and lightweight optimized models, thermal camera, drone detection.
- ❑ **Long term:** cooperative multi-UAV systems integrated with security infrastructure, with strong cybersecurity and privacy.





THANK YOU